AURA RETAIL OS – SYSTEM DESIGN PROPOSAL (PATH A)

[Group 23]

Members:

Krisha Doshi (202512112)

Maahir Shah (202512022)

Rishika Shah (202512041)

Meet Sheth (202512057)

1. Introduction

Aura Retail OS is designed as a modular and adaptive software platform for autonomous retail kiosks deployed across a smart city. These kiosks operate in diverse environments such as hospitals, metro stations, and emergency zones, each requiring different operational policies and behaviors.

The proposed system follows an Adaptive Autonomous System (Path A) approach, enabling the kiosk to dynamically adjust its behavior based on system conditions such as demand, failures, and emergency situations.

2. Problem Understanding

The previous version of the system suffered from multiple limitations:

- Monolithic architecture made updates difficult
- Hardware changes required significant code modifications
- Payment systems were incompatible across vendors
- Lack of consistent security and validation mechanisms
- System failures led to inconsistent states
- No support for dynamic feature addition

To address these issues, the new system focuses on modularity, adaptability, and low coupling between components.

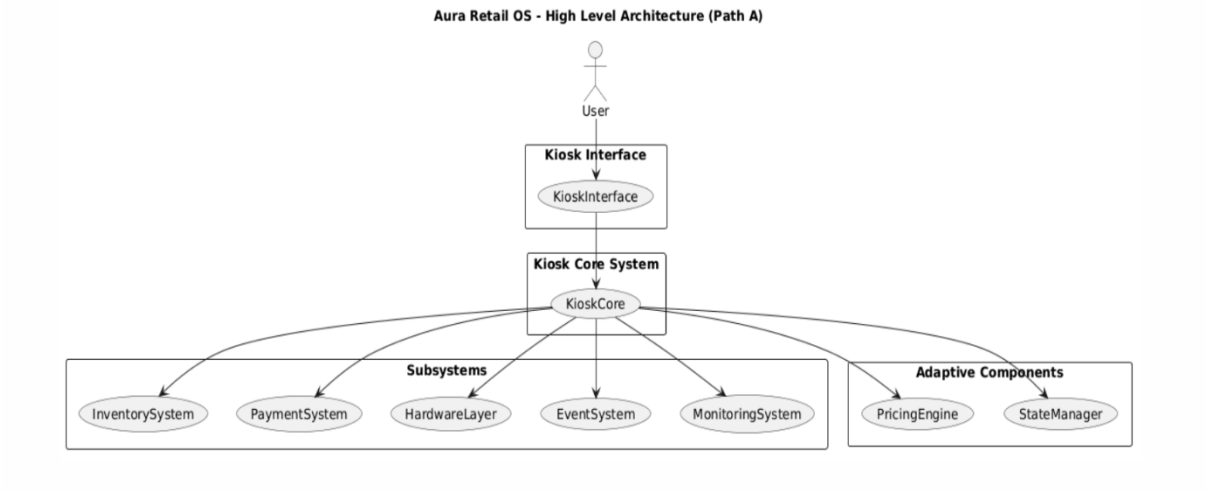
3. System Overview

The Aura Retail OS is divided into multiple subsystems, each responsible for a specific functionality:

- **Kiosk Core System**: Handles user interaction and coordinates all operations
- **Inventory System**: Manages product availability and stock levels
- **Payment System**: Processes transactions through different payment providers
- **Hardware Abstraction Layer**: Interfaces with physical components like dispensers and sensors
- **Event System**: Enables communication between subsystems using events
- **City Monitoring System**: Tracks system status and failures

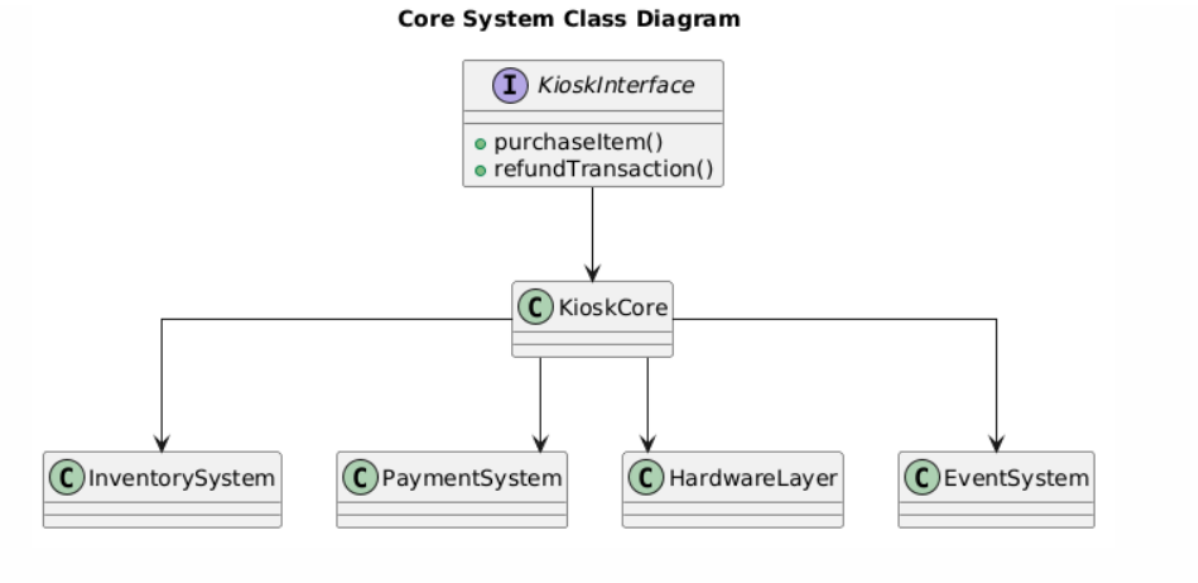
The system ensures that all components interact through well-defined interfaces to maintain low coupling and high scalability.

4. High-Level Architecture Diagram



Explanation:
The architecture follows a centralized coordination model where the Kiosk Core interacts with all subsystems. An event-driven approach is used to ensure loose coupling between components, allowing the system to react dynamically to changes such as failures or emergency conditions.

5. Core Class Diagram



Explanation:
The core class diagram represents the primary structure of the system, including the Kiosk Interface, Kiosk Core, and major subsystems such as Inventory, Payment, Hardware, and Event System. These components form the foundation of the system.

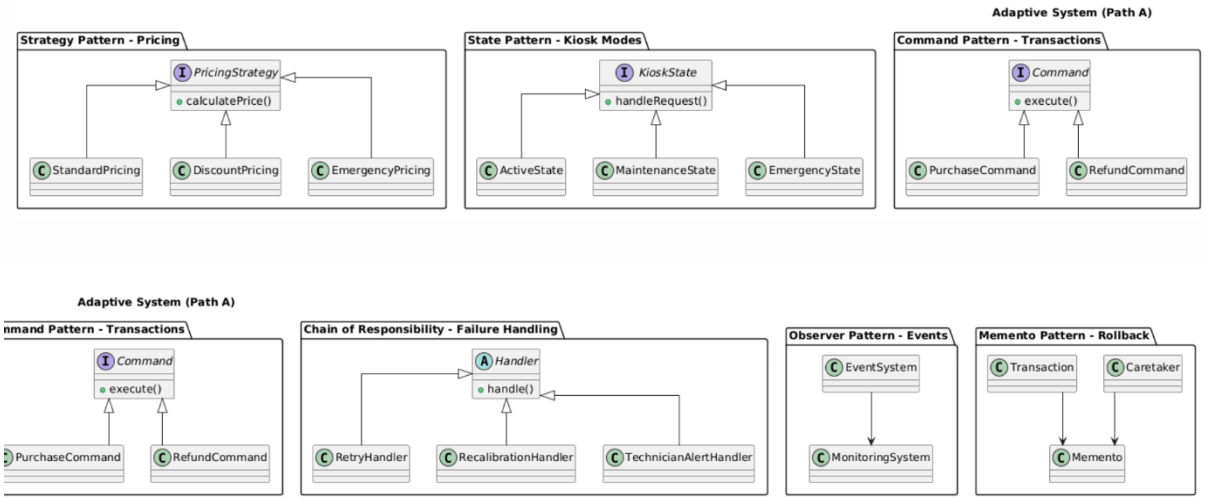
6. Adaptive System Design (Path A)

The system incorporates adaptive behavior to handle dynamic real-world conditions:

- **Dynamic Pricing System**
  Pricing policies such as standard, discount, and emergency pricing are applied based on context.

- **Kiosk Operational Modes**
  The kiosk operates in different modes such as Active, Maintenance, and Emergency, each altering system behavior.
- **Failure Handling System**
  Failures are handled through multiple recovery steps like retry, recalibration, and technician alerts.
- **Event Notification System**
  Subsystems communicate through events, ensuring flexibility and scalability.
- **Transaction Rollback**
  The system restores previous states in case of transaction failure to maintain consistency.

7. Adaptive Class Diagram



Explanation:
This diagram highlights the behavioral components of the system, including pricing strategies, state management, command execution, event handling, and failure recovery mechanisms.

8. Planned Design Patterns

Strategy Pattern

Used for implementing dynamic pricing by allowing the system to switch between different pricing strategies at runtime.

State Pattern

Used to manage kiosk operational modes such as active, maintenance, and emergency.

Command Pattern

Used to encapsulate user operations like purchase and refund as command objects.

Observer Pattern

Used for event-driven communication between subsystems.

Chain of Responsibility

Used for handling failures through a sequence of recovery steps.

Memento Pattern

Used to implement transaction rollback and restore previous system states.

Abstract Factory Pattern

Used to create different types of kiosks with compatible components.

9. USP

To differentiate our system from standard implementations, we introduce an **Intelligence Layer** that enhances system adaptability:

- **Predictive Pricing Mechanism**
  Prices are adjusted based on demand, time, and system conditions.
- **Self-Healing Failure Recovery**
  The system attempts automatic recovery before escalating failures.
- **Priority-Based Event Handling**
  Critical events such as emergency activation override normal operations instantly.

This intelligent layer enables the system to go beyond reactive behavior and become proactive and adaptive.

10. Subsystem Responsibilities

The system is divided among four team members to ensure modular development:

- **Member 1**: Krisha Doshi (202512112)

  Kiosk Core System and State Management

- **Member 2**: Maahir Shah (202512022)

  Inventory System and stock management

- **Member 3**: Rishika Shah (202512041)

  Payment and Transaction System

- **Member 4**: Meet Sheth (202512057)

  Hardware Layer, Event System, and Failure Handling

11. Conclusion

The proposed Aura Retail OS provides a scalable, modular, and adaptive solution for smart-city retail kiosks. By leveraging object-oriented principles and design patterns, the system ensures flexibility, reliability, and efficient handling of real-world scenarios such as failures and emergencies.